Lesson 1.2 — The Skill Stack

Lesson 1.2 — The Skill Stack

> **Engineering work breaks into five layers. AI is now competent at two of them. You sell the other three.** This lesson is the map.

By the end of this lesson:

- You'll have a five-layer model of engineering work that you can draw on a whiteboard in any interview.
- You'll know which layer each AI tool actually operates at — and where the marketing oversells.
- You'll know the *new* skills that didn't exist five years ago and that command a premium today.
- You'll have a soundbite that converts the question *"Why do we need you?"* into a structural answer.

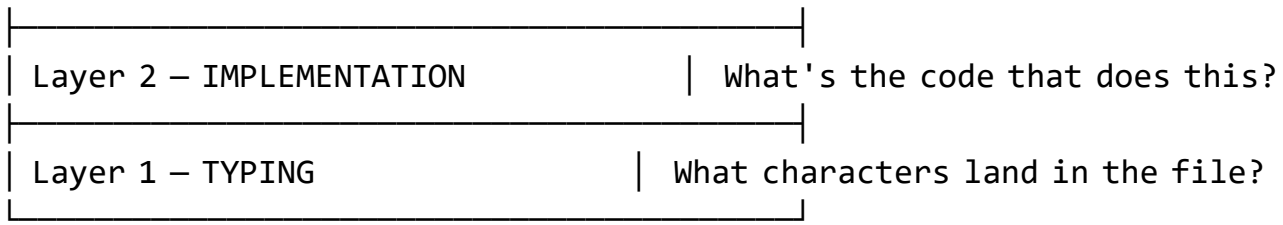
This is the lesson that wins the next chapter for you. Read it twice if you have to.

---

1. The five layers of engineering work

Stack them from concrete to abstract:

```
┌──────────────────────────────────────────────┐
│ Layer 5 — VALUE                              │ Why does this work exist?
├──────────────────────────────────────────────┤
│ Layer 4 — JUDGMENT                           │ Which of these solutions ships?
├──────────────────────────────────────────────┤
│ Layer 3 — DESIGN                             │ How do the pieces fit together?
```



Every task in your day touches one or more of these layers. **Beginners think their job is Layer 1 and 2. Seniors know their job is Layer 3, 4, and 5.** The AI revolution made Layers 1 and 2 cheap. It did *not* make Layers 3, 4, and 5 cheap.

Read that paragraph three times. It is the entire thesis of this course.

---

2. Layer-by-layer breakdown

Layer 1 — Typing

What it is: keystrokes. Pressing letter keys to put characters in a file.

Examples:

- Typing `const user = { id: 1, email: 'a@b.com' };`
- Importing a module.
- Copy-pasting boilerplate.
- Renaming a variable in 12 places.

Who's doing it now? The AI. Almost entirely. Even when you "write" code, the AI completes the line. This was the entire promise of Copilot in 2022 and it's been delivered.

What's left for the human? Choosing what to type. Which is to say: it's not really a layer anymore.

Layer 2 — Implementation

What it is: turning a clear specification into working code.

Examples:

- "Implement a debounce function with a 300ms wait."
- "Add a loading state to this button."
- "Write a Postgres query that returns the top 10 customers by revenue this month."

- "Convert these JSON samples into a TypeScript interface."

Who's doing it now? Mostly the AI, *given a clear spec*. AI is excellent at "I know exactly what I want; produce it."

What's left for the human? Writing the spec. Reviewing the output. Catching the cases the AI didn't handle (off-by-one, locale, timezone, edge case the spec didn't mention).

Layer 3 — Design

What it is: deciding how the pieces fit together.

Examples:

- "Should this state live in the component, the parent, or a global store?"
- "Should this be one endpoint with a query parameter, or two endpoints?"
- "Is this a synchronous transformation or an async pipeline?"
- "Should we cache at the CDN, the server, or the client?"

Who's doing it now? *Mostly* the human, with AI as a sounding board.

The AI can list options. It cannot evaluate them against your team's codebase, deploy story, on-call rotation, customer mix, or last quarter's incident postmortems. **Design is contextual. AI has no context except what you give it.**

What's left for the human? Almost everything that matters. This is where seniority shows.

Layer 4 — Judgment

What it is: deciding *which design is right for our situation*.

Examples:

- "We have three valid approaches. Which one ships?"
- "Is this performance optimization worth the readability cost?"
- "Should we build this ourselves or pull in a library?"
- "Is this a fast hack we'll throw away in 3 months, or is this load-bearing forever?"

Who's doing it now? The human, exclusively.

The AI can articulate trade-offs. It cannot *make* the trade-off. The trade-off requires owning the consequences. The AI does not own consequences. **You do.**

What's left for the human? The decision. And the explanation, in the standup, in the docs, in the postmortem six months from now.

Layer 5 — Value

What it is: deciding *whether to do this work at all*.

Examples:

- "Is this feature worth building?"
- "Is the right move to scale this team or kill the project?"
- "Should we move our infra to a new cloud, or absorb the cost of staying?"
- "Should this go on the roadmap, or should we tell sales no?"

Who's doing it now? Humans, in conversations with other humans.

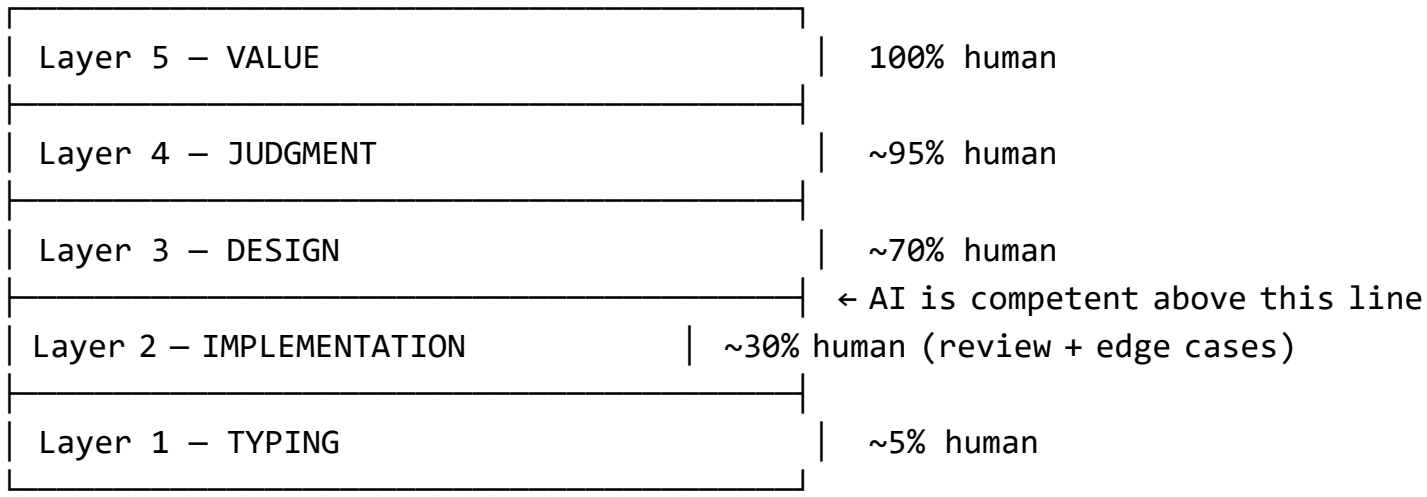
The AI is irrelevant to Layer 5. It can summarize a doc that helps you make the call, but the call itself is yours.

What's left for the human? Everything. Always.

---

3. The "AI replaced developers" myth, in one diagram

Now look at the layers again, with the AI line drawn in:



When pundits say "AI replaced developers," they mean Layer 1 and most of Layer 2. They

are correct. **They are also describing maybe 20% of what a senior engineer's day is actually about.**

The other 80% is Layer 3, 4, and 5. Those layers grew, in fact, because AI made it cheaper to *try* a solution, which created more solutions to evaluate. **More options to choose between more judgment required, not less.**

This is the senior-engineer answer to "AI replaced developers" and it is unfailingly persuasive.

---

4. The new skills (didn't exist 5 years ago, command a premium today)

Five skills are *new*. None of them are in a CS curriculum yet. All of them are interview-tested in 2026.

4.1. Prompt engineering as engineering

Not "prompt hacks." Real engineering: stable system prompts, structured contexts, deterministic output formats, eval suites that score prompts the way unit tests score functions.

> *Interview signal:* "I treat my prompts like code. Version-controlled, with regression evals when I change them."

4.2. Eval-driven development

Writing test cases (sometimes called "evals") that gauge whether an AI-driven feature is doing the right thing. Includes ground-truth datasets, automated scoring, and human-in-the-loop review.

> *Interview signal:* "Before I shipped the AI-summarization feature, I had 50 hand-graded test inputs and a CI job that scored each new prompt version against them."

4.3. AI code review

The skill of reviewing AI-generated code with a different mental model than human-generated code. AI tends to make different mistakes (overconfident plausible-but-wrong

APIs, missing edge cases the spec didn't mention, security mistakes in copy-pasted patterns). Knowing the failure modes is its own discipline.

> *Interview signal:* "AI tends to write code that looks correct but uses APIs that don't exist or are deprecated. My review checklist for AI code is different from my review checklist for human code."

4.4. Cost and latency thinking

LLM calls cost money and add latency. Engineering features that use LLMs at runtime requires understanding token counts, cache strategies, model tiering ("fast model for the easy cases, smart model for the hard ones"), and graceful degradation.

> *Interview signal:* "We use Claude Haiku for the first pass and only escalate to Sonnet if the response fails our quality check. That cut our LLM bill 70% and average latency 40%."

4.5. Risk and safety thinking

Knowing where AI must not be in the loop. PII redaction. Prompt-injection defense. Output validation. Regulated-industry constraints.

> *Interview signal:* "We never let user content reach the model raw — we run a redaction pass first, and we treat any tool the model can call as if a malicious user had typed it."

---

5. The skills you can stop selling

Be honest. Some skills you used to sell on resumes are now AI-trivial:

- "I write clean code." (AI writes acceptable code in milliseconds.)
- "I know syntax X, Y, Z." (AI knows syntax X, Y, Z and 200 others.)
- "I implement features quickly." (AI implements them faster.)
- "I write good test coverage." (AI generates tests endlessly.)
- "I refactor for readability." (AI refactors on command.)

This doesn't mean those skills are worthless. It means **selling them in 2026 is a junior signal**. Senior candidates lead with Layer 3, 4, and 5 stories.

6. The pivot move: convert resume bullets

Take a resume bullet that's Layer-1-or-2-flavored and rewrite it to highlight Layer-3-or-higher work.

Before	After
"Implemented dashboard with 12 charts in React."	"Designed dashboard architecture for 12 chart types; chose Recharts over D3 because the team's React fluency was higher than its SVG fluency."
"Reduced bundle size by 40%."	"Identified bundle bloat as a Lighthouse-driven product priority, traced it to a charting library import, and lobbied for a code-split that traded one network round-trip for 40% smaller initial bundle."
"Wrote 90% test coverage on payments module."	"Set the testing strategy for the payments module, focused integration tests on the cross-cutting failure modes (refunds, partial captures, currency rounding), and made the trade-off explicit in the team doc."

Notice the pattern. The "after" version names a *decision* and a *constraint*. That's how you signal Layer 3+. AI cannot make those bullets for you because AI didn't make those decisions; you did.

---

7. The interview soundbite

When the interviewer asks the loaded version — *"Aren't you just doing what AI does now?"* — you reach for this:

> *"There are five layers in any engineering task. AI is genuinely good at the bottom two: typing and clear-spec implementation. The top three — design choices that fit the team's context, judgment about which trade-off ships, and value-level decisions about what to build at all — those are still human, and the AI hasn't moved on them. The reason I'm useful isn't that I type faster than the AI. It's that I make the design choices that decide whether the AI's typing was worth doing."*

That's ~70 words. It fits anywhere. **Drill it.**

---

8. Common mistakes engineers make in this conversation

8.1. Conceding too much to AI

> *"I mean, AI does most of my work now."*

You just told the interviewer they don't need you. **Don't say this even if you privately feel it.** What's actually true: AI does most of your *typing*. Your *work* is upstream of typing.

8.2. Conceding too little

> *"AI is overhyped, I barely use it."*

You just told the interviewer you're slower than the AI-fluent candidate they'll see this afternoon. Don't say this either.

8.3. The "everything is the same" lie

> *"Honestly, my job is the same as it was three years ago."*

It isn't. The interviewer knows it isn't. Saying it is signals you haven't been paying attention.

8.4. The "I'm a manager now" cop-out

> *"I just review the AI's code now, like a manager."*

Two problems. First, "I review code" is itself a craft skill, not a desk job. Second, the interviewer hires engineers, not managers. **Talk about the engineering judgment in the review, not just the act of reviewing.**

---

9. CHECK YOURSELF

- ☐ Can you draw the 5-layer model from memory?
- ☐ Can you name two examples per layer of work that lives there?
- ☐ Can you name three tasks AI is good at and three it's bad at, and assign each to a layer?
- ☐ Can you list the 5 new skills, with one concrete example apiece?
- ☐ Can you rewrite a real resume bullet of yours to lift it from Layer 2 to Layer 3?
- ☐ Can you deliver the section-7 soundbite without notes?

---

10. Where you are now

You have:

- A defendable model that puts AI's contribution in proportion.
- The vocabulary to push interviewers' framing back to your strengths.
- A list of new skills you can name and prove.
- An exercise (resume rewrites) that turns this lesson into immediate, practical updates to your portfolio.

Move on to **03-the-mindset-shift.md** — the lesson that turns this conceptual model into an *attitude* the interviewer can feel from the first 30 seconds of conversation.